



Minimum Subset Sum Difference (hard)

We'll cover the following



- Problem Statement
 - Example 1:
 - Example 2:
 - Example 3:
- Try it yourself
- Basic Solution
 - Code
 - Time and Space complexity
- Top-down Dynamic Programming with Memoization
 - Code
 - Bottom-up Dynamic Programming
 - Code
 - Time and Space complexity

Problem Statement

Given a set of positive numbers, partition the set into two subsets with minimum difference between their subset sums.

Example 1:

Input: {1, 2, 3, 9}

Output: 3

Explanation: We can partition the given set into two subsets where minimum absolute difference between the sum of numbers is '3'. Following are the two subsets: {1, 2, 3} & {9}.

Example 2:

Input: {1, 2, 7, 1, 5}

Output: 0

Explanation: We can partition the given set into two subsets where minimum absolute difference between the sum of number is '0'. Following are the two subsets: {1, 2, 5} & {7, 1}.

Example 3:

Input: {1, 3, 100, 4}

Output: 92



Explanation: We can partition the given set into two subsets where minimum absolute difference between the sum of numbers is '92'. Here are the two subsets: {1, 3, 4} & {100}.

Try it yourself



Try solving this question here:

Java
 Python3
 C++
 JS

```

1  class PartitionSet {
2
3      public int canPartition(int[] num) {
4          // TODO: Write your code here
5          return -1;
6      }
7
8      public static void main(String[] args) {
9          PartitionSet ps = new PartitionSet();
10         int[] num = {1, 2, 3, 9};
11         System.out.println(ps.canPartition(num));
12         num = new int[]{1, 2, 7, 1, 5};
13         System.out.println(ps.canPartition(num));
14         num = new int[]{1, 3, 100, 4};
15         System.out.println(ps.canPartition(num));
16     }
17 }
  
```

Run
 Save
 Reset

Basic Solution

This problem follows the **0/1 Knapsack pattern** and can be converted into a [Subset Sum](#) problem.

Let's assume S1 and S2 are the two desired subsets. A basic brute-force solution could be to try adding each element either in S1 or S2 in order to find the combination that gives the minimum sum difference between the two sets.

So our brute-force algorithm will look like:

```

1  for each number 'i'
2      add number 'i' to S1 and recursively process the remaining numbers
3      add number 'i' to S2 and recursively process the remaining numbers
4  return the minimum absolute difference of the above two sets
  
```

Code

Here is the code for the brute-force solution:

Java
 Python3
 C++
 JS

```

1  class PartitionSet {
2
3      public int canPartition(int[] num) {
4          return this.canPartitionRecursive(num, 0, 0, 0);
5      }
6
7      private int canPartitionRecursive(int[] num, int currentIndex, int sum1, int sum2) {
8          // base check
9
10         // recursive case
11         // if we have reached the end of the array, return the absolute difference
12         // between the two sums
13         if (currentIndex == num.length) {
14             return Math.abs(sum1 - sum2);
15         }
16         // try adding the current element to either subset
17         int sum1WithCurrent = sum1 + num[currentIndex];
18         int sum2WithCurrent = sum2 + num[currentIndex];
19         int sum1WithoutCurrent = sum1;
20         int sum2WithoutCurrent = sum2;
21         // recursive call for the next element
22         int sum1AndSum2WithCurrent = canPartitionRecursive(num, currentIndex + 1, sum1WithCurrent, sum2WithCurrent);
23         int sum1AndSum2WithoutCurrent = canPartitionRecursive(num, currentIndex + 1, sum1WithoutCurrent, sum2WithoutCurrent);
24         // return the minimum of the two recursive calls
25         return Math.min(sum1AndSum2WithCurrent, sum1AndSum2WithoutCurrent);
26     }
27 }
  
```

```

8 // base check
9 if (currentIndex == num.length)
10     return Math.abs(sum1 - sum2);
11
12 // recursive call after including the number at the currentIndex in the first set
13 int diff1 = canPartitionRecursive(num, currentIndex+1, sum1+num[currentIndex], sum2);
14
15 // recursive call after including the number at the currentIndex in the second set
16 int diff2 = canPartitionRecursive(num, currentIndex+1, sum1, sum2+num[currentIndex]);
17
18 return Math.min(diff1, diff2);
19 }
20
21 public static void main(String[] args) {
22     PartitionSet ps = new PartitionSet();
23     int[] num = {1, 2, 3, 9};
24     System.out.println(ps.canPartition(num));
25     num = new int[]{1, 2, 7, 1, 5};
26     System.out.println(ps.canPartition(num));
27     num = new int[]{1, 3, 100, 4};
28     System.out.println(ps.canPartition(num));
29 }
30 }

```

Run

Save

Reset



Time and Space complexity

Because of the two recursive calls, the time complexity of the above algorithm is exponential $O(2^n)$, where 'n' represents the total number. The space complexity is $O(n)$ which is used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems.

We will be using a two-dimensional array to store the results of the solved sub-problems. We can uniquely identify a sub-problem from 'currentIndex' and 'Sum1' as 'Sum2' will always be the sum of the remaining numbers.

Code

Here is the code:

Java

Python3

C++

JS

```

1 class PartitionSet {
2
3     public int canPartition(int[] num) {
4         int sum = 0;
5         for (int i = 0; i < num.length; i++)
6             sum += num[i];
7
8         Integer[][] dp = new Integer[num.length][sum + 1];
9         return this.canPartitionRecursive(dp, num, 0, 0, 0);
10    }
11
12    private int canPartitionRecursive(Integer[][] dp, int[] num, int currentIndex, int sum1, int sum2) {
13        // base check
14        if (currentIndex == num.length)
15            return Math.abs(sum1 - sum2);
16
17        // check if we have not already processed similar problem

```

```

18     if(dp[currentIndex][sum1] == null) {
19         // recursive call after including the number at the currentIndex in the first set
20         int diff1 = canPartitionRecursive(dp, num, currentIndex + 1, sum1 + num[currentIndex], sum2);
21
22         // recursive call after including the number at the currentIndex in the second set
23         int diff2 = canPartitionRecursive(dp, num, currentIndex + 1, sum1, sum2 + num[currentIndex]);
24
25         dp[currentIndex][sum1] = Math.min(diff1, diff2);
26     }
27
28     return dp[currentIndex][sum1];
29 }
30
31 public static void main(String[] args) {
32     PartitionSet ps = new PartitionSet();
33     int[] num = {1, 2, 3, 9};
34     System.out.println(ps.canPartition(num));
35     num = new int[]{1, 2, 7, 1, 5};
36     System.out.println(ps.canPartition(num));
37     num = new int[]{1, 3, 100, 4};
38     System.out.println(ps.canPartition(num));
39 }
40 }

```

Run

Save

Reset



Bottom-up Dynamic Programming

Let's assume 'S' represents the total sum of all the numbers. So, in this problem, we are trying to find a subset whose sum is as close to 'S/2' as possible, because if we can partition the given set into two subsets of an equal sum, we get the minimum difference, i.e. zero. This transforms our problem to [Subset Sum](#), where we try to find a subset whose sum is equal to a given number-- 'S/2' in our case. If we can't find such a subset, then we will take the subset which has the sum closest to 'S/2'. This is easily possible, as we will be calculating all possible sums with every subset.

Essentially, we need to calculate all the possible sums up to 'S/2' for all numbers. So how can we populate the array `dp[TotalNumbers][S/2+1]` in the bottom-up fashion?

For every possible sum 's' (where $0 \leq s \leq S/2$), we have two options:

1. Exclude the number. In this case, we will see if we can get the sum 's' from the subset excluding this number => `dp[index-1][s]`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum => `dp[index-1][s-num[index]]`

If either of the two above scenarios is true, we can find a subset with a sum equal to 's'. We should dig into this before we can learn how to find the closest subset.

Let's draw this visually, with the example input {1, 2, 3, 9}. Since the total sum is '15', we will try to find a subset whose sum is equal to the half of it, i.e. '7'.



num\sum	0	1	2	3	4	5	6	7
1	T							
{1, 2}	T							
{1,2,3}	T							
{1,2,3,9}	T							

'0' sum can always be found through an empty set

1 of 11



The above visualization tells us that it is not possible to find a subset whose sum is equal to '7'. So what is the closest subset we can find? We can find the subset if we start moving backwards in the last row from the bottom right corner to find the first 'T'. The first "T" in the diagram above is the sum '6', which means that we can find a subset whose sum is equal to '6'. This means the other set will have a sum of '9' and the minimum difference will be '3'.

Code

Here is the code for our bottom-up dynamic programming approach:



```

1 class PartitionSet {
2
3     public int canPartition(int[] num) {
4         int sum = 0;
5         for (int i = 0; i < num.length; i++)
6             sum += num[i];
7
8         int n = num.length;
9         boolean[][] dp = new boolean[n][sum/2 + 1];
10
11         // populate the sum=0 columns, as we can always form '0' sum with an empty set
12         for(int i=0; i < n; i++)
13             dp[i][0] = true;
14
15         // with only one number, we can form a subset only when the required sum is equal to that number
16         for(int s=1; s <= sum/2 ; s++) {
17             dp[0][s] = (num[0] == s ? true : false);
18         }
19
20         // process all subsets for all sums
21         for(int i=1; i < num.length; i++) {
22             for(int s=1; s <= sum/2; s++) {
23                 // if we can get the sum 's' without the number at index 'i'
24                 if(dp[i-1][s]) {
25                     dp[i][s] = dp[i-1][s];
26                 } else if (s >= num[i]) {
27                     // else include the number and see if we can find a subset to get the remaining sum
28                     dp[i][s] = dp[i-1][s-num[i]];
29                 }
30             }
31         }
32     }
33 }

```

```
31     }
32
33     int sum1 = 0;
34     // Find the largest index in the last row which is true
35     for (int i = sum / 2; i >= 0; i--) {
36         if (dp[n-1][i] == true) {
37             sum1 = i;
38             break;
39         }
40     }
41
42     int sum2 = sum - sum1;
43     return Math.abs(sum2-sum1);
44 }
45
46 public static void main(String[] args) {
47     PartitionSet ps = new PartitionSet();
48     int[] num = {1, 2, 3, 9};
49     System.out.println(ps.canPartition(num));
50     num = new int[]{1, 2, 7, 1, 5};
51     System.out.println(ps.canPartition(num));
52     num = new int[]{1, 3, 100, 4};
53     System.out.println(ps.canPartition(num));
54 }
55 }
```

[Run](#)[Save](#)[Reset](#)

Time and Space complexity

The above solution has the time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

[← Back](#)☒ [Mark As Completed](#)[Next →](#)